RAG TECHNIQUE

Interview Question Bank

Medium → Advanced Scenario-Based Questions

Retrieval-Augmented Generation | AI Engineer Edition | 2025–2026

31 Questions · 8 Topic Areas · Scenario + Hint Format

About This Question Bank

This document contains medium-to-advanced level, scenario-based interview questions on Retrieval-Augmented Generation (RAG). Each question places the candidate in a realistic production situation, mirroring how top AI engineering interviews are structured in 2025–2026.

Questions are organised into 8 topic areas. Each question is accompanied by a 'Expected Coverage' hint that lists the key concepts, trade-offs, and tools the ideal answer should address.

Who Should Use This

- AI / ML Engineers preparing for technical interviews
- Senior engineers moving into GenAI / LLM roles
- Interviewers and hiring managers building RAG-focused assessment rubrics
- Data scientists expanding into production LLM system design

Topic Areas Covered

System Design & Architecture

Q1. Your team is building a customer support chatbot for a SaaS platform that handles 50,000 daily queries. The knowledge base contains 200,000+ support articles, FAQs, and release notes. How would you design the RAG pipeline from scratch?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Vector DB selection (Pinecone/Weaviate/Qdrant), chunking strategy, embedding model choice, hybrid retrieval, caching layer, latency SLAs, monitoring hooks.

Q2. A fintech company wants a RAG-based compliance assistant that answers regulatory queries by referencing RBI, SEBI, and internal policy documents — all of which are updated monthly. How do you handle knowledge freshness without re-indexing the entire corpus?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Incremental indexing, document versioning, staleness metadata, TTL-based invalidation, change-detection pipelines, audit trails.

Q3. You need to design a multi-tenant RAG system where each enterprise client has isolated data but the underlying LLM and vector infrastructure is shared. What architectural choices ensure data isolation without sacrificing performance?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Namespace partitioning in vector DBs, per-tenant embedding namespaces, role-based retrieval guards, API-layer tenant context injection, resource quotas.

Q4. An e-commerce company wants a product recommendation and Q&A system. Their catalog has structured data (PostgreSQL), product images, and unstructured review documents. How do you design a multimodal RAG system that leverages all three?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Multimodal embeddings (CLIP/LLaVA), cross-modal retrieval, structured vs. unstructured index fusion, late fusion strategy, query routing by modality.

Chunking & Indexing Strategy

Q5. You are indexing a 500-page legal contract. Fixed-size chunking at 512 tokens is causing retrieval failures because clause boundaries are split mid-sentence. How do you redesign the chunking strategy, and how do you validate the improvement?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Recursive character splitting, semantic chunking, sentence-boundary awareness, parent-child (hierarchical) chunking, chunk overlap tuning, retrieval precision@K evaluation.

Q6. After deploying a RAG system, your team notices that answers to questions spanning multiple sections of a document (e.g., 'What is the total budget breakdown across all departments?') are consistently incomplete. What is the root cause and how do you fix it?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Multi-hop retrieval failure, sentence-window chunking, RAPTOR (recursive summarisation), late chunking, parent-document retriever pattern, query decomposition.

Q7. Your RAG system indexes both dense technical manuals (10,000+ words) and short FAQ entries (50–100 words). A uniform chunk size produces poor retrieval for both. How do you handle documents with highly variable length and structure?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Document-type-aware chunking, metadata tagging by document type, adaptive chunk sizing, hierarchical indexing, per-collection chunking policies.

Q8. A new embedding model outperforms your current one on benchmarks, but re-embedding 5 million chunks would cost significant compute and time. How do you plan and execute an embedding model migration in production with minimal downtime and risk?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Dual-index blue-green deployment, shadow retrieval comparison, batch migration with rollback triggers, embedding drift monitoring, A/B traffic splitting.

Retrieval Quality & Ranking

Q9. Users of your RAG system report that exact keyword queries (e.g., 'Invoice #4521 status') return poor results, while semantic queries work well. Your current system uses only dense vector retrieval. How do you address this?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Hybrid search (BM25 + dense), reciprocal rank fusion (RRF), weighted score combination, sparse + dense index (SPLADE/BM25), keyword-aware query routing.

Q10. You observe that retrieving top-20 chunks instead of top-5 improves recall but degrades generation quality — the LLM starts ignoring relevant chunks or hallucinating. How do you balance recall vs. context precision?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Cross-encoder reranking, Maximal Marginal Relevance (MMR) for diversity, context compression/summarisation, adaptive top-K, lost-in-the-middle mitigation, relevance score thresholds.

Q11. A RAG system for a healthcare provider retrieves outdated clinical guidelines even though the latest versions are indexed. Investigation reveals the stale chunks have richer semantic content and score higher on cosine similarity. How do you solve this?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Metadata freshness filters (date-gated retrieval), recency boosting in scoring, treating version metadata as a hard filter pre-retrieval, source authority weighting.

Q12. Users ask multi-part questions like: 'Compare the return policy of Product A with Product B, and tell me which is more customer-friendly.' Your current single-retrieval pipeline fails here. How do you redesign retrieval?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Query decomposition (sub-query generation), parallel retrieval per sub-query, result merging and deduplication, LLM-orchestrated multi-step retrieval, agentic tool use.

Hallucination Control & Factual Accuracy

Q13. Your RAG system for a legal firm confidently answers a question about case law with citations that seem plausible but are fabricated. The retrieved chunks were irrelevant but the LLM used its parametric knowledge. How do you detect and prevent this?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Citation grounding enforcement in system prompt, faithfulness scoring (RAGAS), retrieved-vs-generated attribution check, fallback 'I don't know' instruction, NLI-based hallucination detection post-generation.

Q14. During evaluation, you find that your RAG system produces correct answers 85% of the time, but 10% of wrong answers include highly confident false citations. What metrics and guardrails do you put in place before re-deploying?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: RAGAS faithfulness/answer relevance/context precision, Trulens evaluation, citation attribution pipeline, confidence-calibration, human-in-the-loop escalation triggers.

Q15. A RAG pipeline for a medical diagnosis support tool sometimes mixes context from two different patient case studies during generation, producing a blended but incorrect summary. What causes this and how do you prevent it?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Context window contamination, poor chunk boundaries, missing patient ID metadata filters, per-session context isolation, strict metadata-gated retrieval, structural prompt separation.

Q16. After switching from GPT-4 to a smaller open-source LLM to reduce costs, hallucination rates increase even though retrieval quality remains the same. What is the relationship between generator model quality and hallucination in RAG, and how do you adapt?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Generator's ability to stay grounded to context, instruction-following capability, prompt tuning for context adherence, RAFT (Retrieval-Augmented Fine-Tuning), output validation layer.

Advanced RAG Architectures

Q17. Your RAG system is being upgraded to a Self-RAG architecture. Walk through how Self-RAG differs from naive RAG, what the reflection tokens ('Retrieve', 'ISREL', 'ISSUP', 'ISUSE') do, and in which production scenarios you would choose Self-RAG over standard RAG.

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: On-demand retrieval decision, per-segment relevance scoring, iterative refinement loop, higher quality at cost of latency, best for open-domain QA with variable retrieval need.

Q18. A knowledge management platform for a pharmaceutical company needs to answer questions that require connecting facts across 50+ research papers (e.g., 'Which drug trials showed adverse effects in patients with kidney disease?'). Why does standard vector RAG fail here, and how does GraphRAG address it?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Multi-hop reasoning across documents, entity-relationship extraction, community detection, hierarchical graph summaries, Microsoft GraphRAG architecture, LazyGraphRAG cost optimization.

Q19. You're designing an Agentic RAG system for a data analyst who can query Salesforce CRM, internal Confluence wikis, and live web search simultaneously. How do you architect the agent loop, tool selection, and retrieval orchestration? What are the failure modes?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: LangGraph/CrewAI agent framework, tool-use planning, retrieval as a tool call, MCP protocol integration, looping termination criteria, tool hallucination risk, latency compounding, observability.

Q20. You are implementing Corrective RAG (CRAG) for an enterprise search product. Explain the evaluation-correction-retrieval loop, and describe specifically when CRAG would trigger a web search fallback vs. trust the indexed knowledge base.

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Relevance evaluator (LLM-as-judge), confidence thresholds, ambiguous zone handling, knowledge base coverage gap detection, web search as supplemental retrieval, result fusion.

Q21. A client wants an Adaptive RAG system that routes queries based on complexity. Describe how you would build the query classifier, define the routing tiers (naive RAG, advanced RAG, agentic RAG), and measure whether routing decisions are correct.

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Query complexity classification (few-shot or trained classifier), 3-tier routing, cost-quality tradeoff per tier, offline routing accuracy measurement, runtime fallback on tier failure.

Production, Scalability & Latency

Q22. Your RAG API is experiencing P95 latency of 4.2 seconds, which violates your SLA of under 2 seconds. Profiling shows 60% of time is spent in the retrieval stage. What optimizations do you apply and in what order?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: ANN algorithm tuning (HNSW ef parameters), query embedding caching, pre-filtering with metadata to shrink search space, approximate vs. exact search tradeoff, async retrieval + generation pipelining, edge vector DB deployment.

Q23. You are scaling a RAG system from 1,000 to 10 million indexed documents. What bottlenecks do you expect at each stage of the pipeline (ingestion, indexing, retrieval, generation) and how do you address each?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Batch ingestion pipelines (Kafka/Spark), distributed vector index sharding, segment-level index management, HNSW vs IVF_PQ tradeoffs at scale, read replicas for retrieval, LLM request batching.

Q24. Your RAG system has been running in production for 6 months. Team members report a gradual 'quality drift' — responses that were accurate 6 months ago are now less reliable. You haven't changed the codebase. What are the likely causes and how do you diagnose them?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Knowledge base staleness (documents not updated), embedding model drift, LLM API version changes (silent model updates), data distribution shift in user queries, index fragmentation over time.

Q25. A real-time RAG system for a stock market news assistant must retrieve and answer within 300ms. Standard cross-encoder reranking alone costs 200ms. How do you maintain retrieval quality while hitting the latency target?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Lightweight bi-encoder reranking, pre-computed reranking scores for common queries, query caching, approximate reranking on top-5 instead of top-20, distilled cross-encoder, async streaming generation.

Evaluation & Observability

Q26. Your team is debating whether the RAG system's quality improvements after adding a cross-encoder reranker are statistically significant. How do you set up a rigorous offline evaluation framework to measure this?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Golden dataset creation (human-labelled QA pairs), RAGAS metrics (faithfulness, answer relevance, context precision/recall), A/B evaluation, statistical significance testing, NDCG and MRR for retrieval.

Q27. Production logs show that 15% of user queries produce no useful answer ('I don't know' or hallucinated responses), but you cannot identify which pipeline stage is failing. How do you build end-to-end observability for a RAG system?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Per-request tracing (query → embedding → top-K chunks + scores → prompt → response), LLM observability tools (LangSmith, Langfuse, Arize), faithfulness scoring on sampled outputs, retrieval recall monitoring, alerting on score degradation.

Q28. The business wants a single KPI dashboard to track the RAG system's quality over time. What metrics would you include, how would you compute them, and what thresholds would trigger an alert?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Retrieval: context precision, context recall, NDCG@K. Generation: faithfulness, answer relevance, hallucination rate. Business: user satisfaction, escalation rate, session deflection rate. Alert thresholds by metric.

Security, Privacy & Governance

Q29. A RAG system for an HR department contains sensitive employee performance reviews and compensation data. A malicious user crafts a prompt to bypass the retrieval filter and extract another employee's records. How do you defend against prompt injection and data exfiltration?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Row-level security in vector DB (metadata ACL filter), user identity propagation to retrieval layer, system prompt hardening against injection, output scanning for PII, query sanitisation, audit logging.

Q30. Your RAG system is deployed for a European healthcare company subject to GDPR. A patient requests erasure of all their data. How do you implement 'right to be forgotten' in a RAG system that has already indexed their records?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Document-level deletion APIs in vector DB (Weaviate/Qdrant support), chunk-level UUID tracking per source document, LLM parametric memory (cannot be erased — risk disclosure), audit trail for deletion verification.

Q31. An internal investigation reveals that indirect prompt injection — malicious instructions embedded in indexed documents — is causing the RAG system to leak internal system prompts and ignore safety instructions. How do you redesign the pipeline to prevent this?

> **Scenario:** *Real-world scenario requiring architectural reasoning, trade-off analysis, and production awareness.*

Expected Coverage: Content scanning at ingestion time, sandboxed retrieval context (strict role separation in prompt), LLM-based injection detection on retrieved chunks, instruction hierarchy enforcement, red-teaming the ingestion pipeline.

Quick Reference: Key RAG Concepts

Term	Definition
Naive RAG	Basic retrieve → generate pipeline with no reranking or correction.
Hybrid Search	Combines dense (vector) + sparse (BM25/TF-IDF) retrieval for keyword + semantic coverage.
Cross-Encoder Reranking	Reranks top-K retrieved chunks using a more expensive but precise model.
MMR (Maximal Marginal Relevance)	Selects diverse chunks to avoid redundancy in the context window.
Self-RAG	Uses reflection tokens to decide when to retrieve and scores its own outputs.
CRAG (Corrective RAG)	Evaluates retrieved documents and triggers fallback web search if quality is low.
GraphRAG	Represents knowledge as entity-relationship graphs for multi-hop reasoning.
Agentic RAG	LLM-orchestrated multi-tool retrieval with planning and self-correction.
RAPTOR	Builds recursive tree-of-summaries for hierarchical document retrieval.
Contextual Retrieval	Prepends chunk-specific context before embedding (Anthropic, 2024).
RAGAS	Evaluation framework: faithfulness, answer relevance, context precision, context recall.
Late Chunking	Embeds the full document first, then chunks — preserving cross-chunk context.
Parent-Child Retrieval	Retrieves small chunks but expands to parent chunk for generation context.
Adaptive RAG	Routes queries to different pipeline tiers based on complexity classification.
Embedding Drift	Degradation in retrieval quality when the underlying embedding model is updated.

— End of Document —

Compiled using current AI engineering interview trends · July 2025